# Practical No.1

```
import java.io.*;
import java.util.*;

class Symbol {
    String symbol;
    int address;

    Symbol(String symbol, int address) {
        this.symbol = symbol;
        this.address = address;
    }
}

class IntermediateLine {
    int address;
    String label;
    String opcode;
    String operand;

    IntermediateLine(int address, String label, String opcode, String operand) {
        this.address = address;
        this.label = label;
        this.opcode = opcode;
        this.operand = operand;
    }

    @Override
    public String toString() {
        return address + "\t" + (label == null ? "-" : label) + "\t" + opcode + "\t" + (operand
== null ? "-" : operand);
    }
}

public class TwoPassAssembler {
    private static Map<String, String> opcodeTable = new HashMap<>();
    private static Map<String, Integer> symbolTable = new HashMap<>();
    private static List<IntermediateLine> intermediateCode = new ArrayList<>();

    private static int locCounter = 0;
```

```
static {
    opcodeTable.put("MOV", "01");
    opcodeTable.put("ADD", "02");
    opcodeTable.put("SUB", "03");
    opcodeTable.put("JMP", "04");
    opcodeTable.put("DC", "DC");
    opcodeTable.put("DS", "DS");
    opcodeTable.put("START", "START");
    opcodeTable.put("END", "END");
}

public static void passOne(List<String> source) {
    locCounter = 0;

    for (String line : source) {
        line = line.trim();
        if (line.isEmpty() || line.startsWith(";")) continue;

        String label = null, opcode = null, operand = null;
        String[] parts = line.split("\\s+");
        if (parts.length == 1) {
            opcode = parts[0];
        } else if (parts.length == 2) {
            opcode = parts[0];
            operand = parts[1];
        } else if (parts.length >= 3) {
            label = parts[0];
            opcode = parts[1];
            operand = parts[2];
        }

        if ("START".equalsIgnoreCase(opcode)) {
            if (operand != null) {
                locCounter = Integer.parseInt(operand);
            }
            intermediateCode.add(new IntermediateLine(locCounter, label, opcode, operand));
            continue;
        }

        if (label != null) {
            if (symbolTable.containsKey(label)) {
```

```
                System.err.println("Error: Duplicate symbol " + label);
            } else {
                symbolTable.put(label, locCounter);
            }
        }

        intermediateCode.add(new IntermediateLine(locCounter, label, opcode, operand));

        if (opcode.equalsIgnoreCase("DC")) {
            locCounter += 1;
        } else if (opcode.equalsIgnoreCase("DS")) {
            int size = operand == null ? 1 : Integer.parseInt(operand);
            locCounter += size;
        } else if (opcode.equalsIgnoreCase("END")) {
            break;
        } else {
            locCounter += 1;
        }
    }
}

public static void passTwo() {
    System.out.println("Address\tMachine Code");
    for (IntermediateLine line : intermediateCode) {
        if (line.opcode.equalsIgnoreCase("START") || line.opcode.equalsIgnoreCase("END"))
{
            continue;
        }

        String mc = "";

        if (opcodeTable.containsKey(line.opcode)) {
            String opCodeHex = opcodeTable.get(line.opcode);

            if (opCodeHex.equals("DC")) {
                mc = String.format("%04X", Integer.parseInt(line.operand));
            } else if (opCodeHex.equals("DS")) {
                int size = line.operand == null ? 1 : Integer.parseInt(line.operand);
                for (int i = 0; i < size; i++) {
                    System.out.printf("%04X\t%04X\n", line.address + i, 0);
                }
                continue;
```

```
                } else {
                    int operandAddress = 0;
                    if (line.operand != null) {
                        if (symbolTable.containsKey(line.operand)) {
                            operandAddress = symbolTable.get(line.operand);
                        } else {
                            System.err.println("Error: Undefined symbol " + line.operand + " at
address " + line.address);
                            operandAddress = 0;
                        }
                    }
                    mc = opCodeHex + String.format("%03X", operandAddress);
                }
            } else {
                System.err.println("Error: Invalid opcode " + line.opcode + " at address " +
line.address);
                mc = "????";
            }

            System.out.printf("%04X\t%s\n", line.address, mc);
        }
    }

    public static void main(String[] args) {
        List<String> sourceProgram = Arrays.asList(
                "START 100",
                "LOOP MOV A",
                "ADD B",
                "SUB C",
                "JMP LOOP",
                "A DC 5",
                "B DC 10",
                "C DC 15",
                "END"
        );

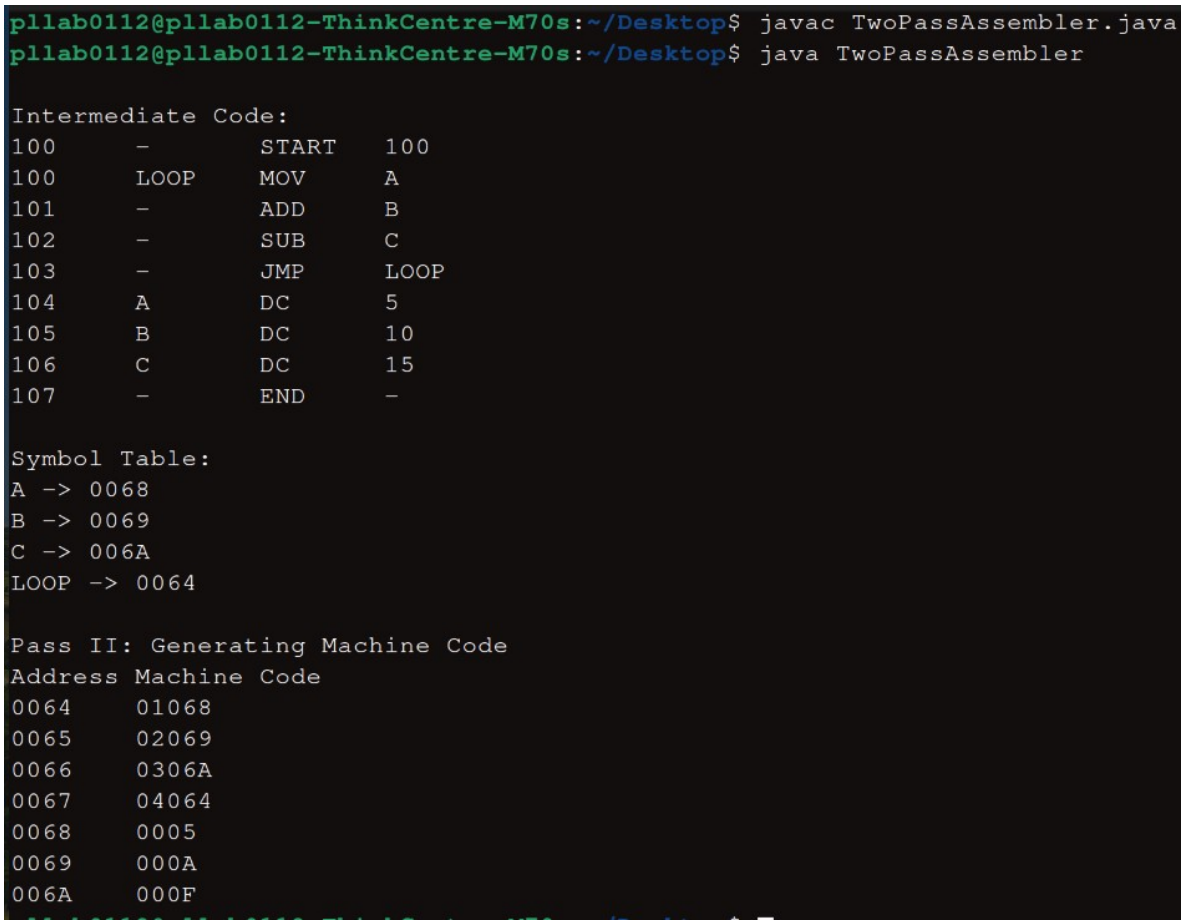
        passOne(sourceProgram);

        System.out.println("\nIntermediate Code:");
        for (IntermediateLine line : intermediateCode) {
            System.out.println(line);
        }
```

```
        System.out.println("\nSymbol Table:");
        for (Map.Entry<String, Integer> entry : symbolTable.entrySet()) {
            System.out.printf("%s -> %04X\n", entry.getKey(), entry.getValue());
        }

        System.out.println("\nPass II: Generating Machine Code");
        passTwo();
    }
}
```

## Output:-



# Practical No. 2

```
import java.io.*;
import java.util.*;

public class TwoPassMacroProcessor {

    static class MNTEntry {
        String name;
        int mdtIndex;
        int paramCount;

        MNTEntry(String name, int mdtIndex, int paramCount) {
            this.name = name;
            this.mdtIndex = mdtIndex;
            this.paramCount = paramCount;
        }
    }

    static List<String> MDT = new ArrayList<>();
    static Map<String, MNTEntry> MNT = new HashMap<>();

    static final String MACRO = "MACRO";
    static final String MEND = "MEND";

    public static void main(String[] args) throws IOException {
        String inputFile = "input.asm";
        String intermediateFile = "intermediate.asm";
        String outputFile = "output.asm";

        passOne(inputFile, intermediateFile);
        passTwo(intermediateFile, outputFile);
    }

    static void passOne(String inputFile, String intermediateFile) throws IOException {
        BufferedReader br = new BufferedReader(new FileReader(inputFile));
        BufferedWriter bw = new BufferedWriter(new FileWriter(intermediateFile));

        String line;
        boolean inMacroDef = false;
        String macroName = null;

        while ((line = br.readLine()) != null) {
            line = line.trim();
            if (line.isEmpty()) continue;

            String[] tokens = line.split("\\s+");

            if (tokens[0].equalsIgnoreCase(MACRO)) {

                line = br.readLine();
                if (line == null) break;
```

```
            line = line.trim();

            String[] headerTokens = line.split("\\s+", 2);
            macroName = headerTokens[0];
            String[] params = new String[0];
            if (headerTokens.length > 1) {
                params = headerTokens[1].split(",");
            }
            inMacroDef = true;

            MNT.put(macroName, new MNTEntry(macroName, MDT.size(), params.length));

            MDT.add(line);
            continue;
        }

        if (inMacroDef) {
            if (tokens[0].equalsIgnoreCase(MEND)) {
                MDT.add(MEND);
                inMacroDef = false;
                continue;
            } else {
                MNTEntry entry = MNT.get(macroName);
                String defLine = line;

                String headerLine = MDT.get(entry.mdtIndex);
                String[] headerParts = headerLine.split("\\s+", 2);

                if (headerParts.length > 1) {
                    String[] paramNames = headerParts[1].split(",");
                    for (int p = 0; p < paramNames.length; p++) {
                        String pname = paramNames[p].trim();
                        if (!pname.isEmpty()) {

                            defLine = defLine.replaceAll("\\b" + pname + "\\b", "#" + (p + 1));
                        }
                    }
                }
                MDT.add(defLine);
            }
        } else {

            bw.write(line);
            bw.newLine();
        }
    }
    br.close();
    bw.close();
}

static void passTwo(String intermediateFile, String outputFile) throws IOException {
    BufferedReader br = new BufferedReader(new FileReader(intermediateFile));
```

```
        BufferedWriter bw = new BufferedWriter(new FileWriter(outputFile));

        String line;
        while ((line = br.readLine()) != null) {
            line = line.trim();
            if (line.isEmpty()) {
                bw.newLine();
                continue;
            }

            String[] tokens = line.split("\\s+");
            String firstToken = tokens[0];

            if (MNT.containsKey(firstToken)) {
                MNTEntry entry = MNT.get(firstToken);


                String argsPart = line.substring(firstToken.length()).trim();
                String[] args = argsPart.isEmpty() ? new String[0] : argsPart.split("\\s*,\\s*");


                int mdtIndex = entry.mdtIndex + 1;
                while (mdtIndex < MDT.size() && !MDT.get(mdtIndex).equalsIgnoreCase(MEND)) {
                    String mdtLine = MDT.get(mdtIndex);


                    for (int i = 0; i < args.length; i++) {
                        mdtLine = mdtLine.replaceAll("\\#" + (i + 1), args[i].trim());
                    }

                    bw.write(mdtLine);
                    bw.newLine();

                    mdtIndex++;
                }
            } else {

                bw.write(line);
                bw.newLine();
            }
        }
        br.close();
        bw.close();
    }
}
```

**input.asm:**

```
MACRO
INCR &A
LOAD &A
ADD ONE
STORE &A
MEND

START
INCR COUNT
END
```

**intermediate.asm:**

```
START
INCR COUNT
END
```

**output.asm:**

```
START
LOAD &A
ADD ONE
STORE &A
END
```

# Practical No. 3

```
import java.util.*;
class Process {
    int pid;
    int burstTime;
    int arrivalTime;
    int remainingTime;
    int waitingTime;
    int turnaroundTime;
    int priority;
    Process(int pid, int burstTime, int arrivalTime) {
        this(pid, burstTime, arrivalTime, 0);
    }
    Process(int pid, int burstTime, int arrivalTime, int priority) {
        this.pid = pid;
        this.burstTime = burstTime;
        this.arrivalTime = arrivalTime;
        this.priority = priority;
        this.remainingTime = burstTime;
    }
}
public class CPUScheduling {
    public static void fcfs(List<Process> processes) {
        System.out.println("\n--- FCFS Scheduling ---");
        int n = processes.size();
        int currentTime = 0;
        int totalWaitingTime = 0;
        int totalTurnaroundTime = 0;
        processes.sort(Comparator.comparingInt(p -> p.arrivalTime));
        for (Process p : processes) {
            if (currentTime < p.arrivalTime) currentTime = p.arrivalTime;
            p.waitingTime = currentTime - p.arrivalTime;
            currentTime += p.burstTime;
            p.turnaroundTime = p.waitingTime + p.burstTime;
            totalWaitingTime += p.waitingTime;
            totalTurnaroundTime += p.turnaroundTime;
        }
        printResults(processes, totalWaitingTime, totalTurnaroundTime);
    }
    public static void sjfNonPreemptive(List<Process> processes) {
        System.out.println("\n--- SJF Scheduling (Non-preemptive) ---");
        int n = processes.size();
        int currentTime = 0, completed = 0;
        int totalWaitingTime = 0, totalTurnaroundTime = 0;
        boolean[] done = new boolean[n];
        while (completed < n) {
            int idx = -1, minBurst = Integer.MAX_VALUE;
            for (int i = 0; i < n; i++) {
                Process p = processes.get(i);
                if (p.arrivalTime <= currentTime && !done[i] && p.burstTime < minBurst) {
                    minBurst = p.burstTime;
                    idx = i;
```

```
            }
        }
        if (idx == -1) {
            currentTime++;
        } else {
            Process p = processes.get(idx);
            p.waitingTime = currentTime - p.arrivalTime;
            currentTime += p.burstTime;
            p.turnaroundTime = p.waitingTime + p.burstTime;
            totalWaitingTime += p.waitingTime;
            totalTurnaroundTime += p.turnaroundTime;
            done[idx] = true;
            completed++;
        }
    }
    printResults(processes, totalWaitingTime, totalTurnaroundTime);
}
public static void sjfPreemptive(List<Process> processes) {
    System.out.println("\n--- SJF Scheduling (Preemptive - SRTF) ---");
    int n = processes.size();
    int completed = 0, currentTime = 0, minIdx = -1;
    int totalWaitingTime = 0, totalTurnaroundTime = 0;
    boolean[] isCompleted = new boolean[n];
    while (completed < n) {
        int minRemaining = Integer.MAX_VALUE;
        minIdx = -1;
        for (int i = 0; i < n; i++) {
            Process p = processes.get(i);
            if (p.arrivalTime <= currentTime && !isCompleted[i] && p.remainingTime <
                minRemaining && p.remainingTime > 0) {
                minRemaining = p.remainingTime;
                minIdx = i;
            }
        }
        if (minIdx == -1) {
            currentTime++;
        } else {
            Process p = processes.get(minIdx);
            p.remainingTime--;
            currentTime++;
            if (p.remainingTime == 0) {
                isCompleted[minIdx] = true;
                completed++;
                p.turnaroundTime = currentTime - p.arrivalTime;
                p.waitingTime = p.turnaroundTime - p.burstTime;
                totalWaitingTime += p.waitingTime;
                totalTurnaroundTime += p.turnaroundTime;
            }
        }
    }
    printResults(processes, totalWaitingTime, totalTurnaroundTime);
}
```

```
public static void roundRobin(List<Process> processes, int timeQuantum) {
    System.out.println("\n--- Round Robin Scheduling (Time Quantum = " + timeQuantum + ")
        ---");
    int n = processes.size();
    int currentTime = 0, completed = 0;
    int totalWaitingTime = 0, totalTurnaroundTime = 0;
    Queue<Process> queue = new LinkedList<>();
    boolean[] inQueue = new boolean[n];
    processes.sort(Comparator.comparingInt(p -> p.arrivalTime));
    int i = 0;
    queue.add(processes.get(i));
    inQueue[i] = true;
    i++;
    while (!queue.isEmpty()) {
        Process p = queue.poll();
        int execTime = Math.min(p.remainingTime, timeQuantum);
        p.remainingTime -= execTime;
        currentTime += execTime;
        for (; i < n; i++) {
            if (processes.get(i).arrivalTime <= currentTime && !inQueue[i]) {
                queue.add(processes.get(i));
                inQueue[i] = true;
            } else {
                break;
            }
        }
        if (p.remainingTime > 0) {
            queue.add(p);
        } else {
            completed++;
            p.turnaroundTime = currentTime - p.arrivalTime;
            p.waitingTime = p.turnaroundTime - p.burstTime;
            totalWaitingTime += p.waitingTime;
            totalTurnaroundTime += p.turnaroundTime;
        }
        if (queue.isEmpty() && completed < n && i < n) {
            queue.add(processes.get(i));
            inQueue[i] = true;
            if (currentTime < processes.get(i).arrivalTime)
                currentTime = processes.get(i).arrivalTime;
            i++;
        }
    }
    printResults(processes, totalWaitingTime, totalTurnaroundTime);
}
public static void priorityScheduling(List<Process> processes) {
    System.out.println("\n--- Priority Scheduling (Non-preemptive) ---");
    int n = processes.size();
    int currentTime = 0, completed = 0;
    int totalWaitingTime = 0, totalTurnaroundTime = 0;
    boolean[] done = new boolean[n];
    while (completed < n) {
```

```
        int idx = -1;
        int highestPriority = Integer.MAX_VALUE;
        for (int i = 0; i < n; i++) {
            Process p = processes.get(i);
            if (p.arrivalTime <= currentTime && !done[i]) {
                if (p.priority < highestPriority) {
                    highestPriority = p.priority;
                    idx = i;
                }
            }
        }
        if (idx == -1) {
            currentTime++;
        } else {
            Process p = processes.get(idx);
            p.waitingTime = currentTime - p.arrivalTime;
            currentTime += p.burstTime;
            p.turnaroundTime = p.waitingTime + p.burstTime;
            done[idx] = true;
            completed++;
            totalWaitingTime += p.waitingTime;
            totalTurnaroundTime += p.turnaroundTime;
        }
    }
    printResults(processes, totalWaitingTime, totalTurnaroundTime);
}
private static void printResults(List<Process> processes, int totalWaitingTime, int t
        otalTurnaroundTime) {
    System.out.println("PID\tArrival\tBurst\tPriority\tWaiting\tTurnaround");
    for (Process p : processes) {
        System.out.printf("%d\t%d\t%d\t%d\t\t%d\t%d\n", p.pid, p.arrivalTime, p.burstTime,
        p.priority, p.waitingTime, p.turnaroundTime);
    }
    System.out.printf("Average Waiting Time: %.2f\n", (float) totalWaitingTime /
        processes.size());
    System.out.printf("Average Turnaround Time: %.2f\n", (float) totalTurnaroundTime /
        processes.size());
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter number of processes: ");
    int n = sc.nextInt();
    List<Process> processes = new ArrayList<>();
    for (int i = 0; i < n; i++) {
        System.out.println("Enter details for process " + (i + 1) + ":");
        System.out.print("Arrival Time: ");
        int arrival = sc.nextInt();
        System.out.print("Burst Time: ");
        int burst = sc.nextInt();
        System.out.print("Priority (lower value = higher priority): ");
        int priority = sc.nextInt();
        processes.add(new Process(i + 1, burst, arrival, priority));
```

```
        }
        System.out.println("\nChoose Scheduling Algorithm:");
        System.out.println("1. FCFS");
        System.out.println("2. SJF (Non-preemptive)");
        System.out.println("3. SJF (Preemptive - SRTF)");
        System.out.println("4. Round Robin");
        System.out.println("5. Priority Scheduling (Non-preemptive)");
        System.out.print("Enter choice: ");
        int choice = sc.nextInt();
        switch (choice) {
            case 1:
                fcfs(cloneProcesses(processes));
                break;
            case 2:
                sjfNonPreemptive(cloneProcesses(processes));
                break;
            case 3:
                sjfPreemptive(cloneProcesses(processes));
                break;
            case 4:
                System.out.print("Enter Time Quantum: ");
                int tq = sc.nextInt();
                roundRobin(cloneProcesses(processes), tq);
                break;
            case 5:
                priorityScheduling(cloneProcesses(processes));
                break;
            default:
                System.out.println("Invalid choice!");
        }
        sc.close();
    }
    private static List<Process> cloneProcesses(List<Process> original) {
        List<Process> clone = new ArrayList<>();
        for (Process p : original) {
            clone.add(new Process(p.pid, p.burstTime, p.arrivalTime, p.priority));
        }
        return clone;
    }
}
```

Output:

```
Enter number of processes: 4
Enter details for process 1:
Arrival Time: 0
Burst Time: 2
Priority (lower value = higher priority): 4
Enter details for process 2:
Arrival Time: 1
Burst Time: 3
Priority (lower value = higher priority): 1
Enter details for process 3:
Arrival Time: 2
Burst Time: 6
Priority (lower value = higher priority): 2
Enter details for process 4:
Arrival Time: 3
Burst Time: 7
Priority (lower value = higher priority): 3

Choose Scheduling Algorithm:
1. FCFS
2. SJF (Non-preemptive)
3. SJF (Preemptive - SRTF)
4. Round Robin
5. Priority Scheduling (Non-preemptive)
Enter choice: 1

--- FCFS Scheduling ---
PID     Arrival Burst   Priority        Waiting Turnaround
1       0       2       4               0       2
2       1       3       1               1       4
3       2       6       2               3       9
4       3       7       3               8       15
Average Waiting Time: 3.00
Average Turnaround Time: 7.50
```

```
Enter number of processes: 4
Enter details for process 1:
Arrival Time: 0
Burst Time: 2
Priority (lower value = higher priority): 4
Enter details for process 2:
Arrival Time: 1
Burst Time: 3
Priority (lower value = higher priority): 1
Enter details for process 3:
Arrival Time: 2
Burst Time: 6
Priority (lower value = higher priority): 2
Enter details for process 4:
Arrival Time: 3
Burst Time: 7
Priority (lower value = higher priority): 3

Choose Scheduling Algorithm:
1. FCFS
2. SJF (Non-preemptive)
3. SJF (Preemptive - SRTF)
4. Round Robin
5. Priority Scheduling (Non-preemptive)
Enter choice: 3

--- SJF Scheduling (Preemptive - SRTF) ---
PID     Arrival Burst   Priority        Waiting Turnaround
1       0       2       4               0       2
2       1       3       1               1       4
3       2       6       2               3       9
4       3       7       3               8       15
Average Waiting Time: 3.00
Average Turnaround Time: 7.50
```

```
Enter number of processes: 4
Enter details for process 1:
Arrival Time: 0
Burst Time: 2
Priority (lower value = higher priority): 4
Enter details for process 2:
Arrival Time: 1
Burst Time: 3
Priority (lower value = higher priority): 1
Enter details for process 3:
Arrival Time: 2
Burst Time: 6
Priority (lower value = higher priority): 2
Enter details for process 4:
Arrival Time: 3
Burst Time: 7
Priority (lower value = higher priority): 3

Choose Scheduling Algorithm:
1. FCFS
2. SJF (Non-preemptive)
3. SJF (Preemptive - SRTF)
4. Round Robin
5. Priority Scheduling (Non-preemptive)
Enter choice: 4
Enter Time Quantum: 3

--- Round Robin Scheduling (Time Quantum = 3) ---
PID     Arrival Burst   Priority        Waiting Turnaround
1       0       2       4               0       2
2       1       3       1               1       4
3       2       6       2               6       12
4       3       7       3               8       15
Average Waiting Time: 3.75
Average Turnaround Time: 8.25
```

```
Enter number of processes: 4
Enter details for process 1:
Arrival Time: 0
Burst Time: 2
Priority (lower value = higher priority): 4
Enter details for process 2:
Arrival Time: 1
Burst Time: 3
Priority (lower value = higher priority): 1
Enter details for process 3:
Arrival Time: 2
Burst Time: 6
Priority (lower value = higher priority): 2
Enter details for process 4:
Arrival Time: 3
Burst Time: 7
Priority (lower value = higher priority): 3

Choose Scheduling Algorithm:
1. FCFS
2. SJF (Non-preemptive)
3. SJF (Preemptive - SRTF)
4. Round Robin
5. Priority Scheduling (Non-preemptive)
Enter choice: 5

--- Priority Scheduling (Non-preemptive) ---
PID     Arrival Burst   Priority        Waiting Turnaround
1       0       2       4               0       2
2       1       3       1               1       4
3       2       6       2               3       9
4       3       7       3               8       15
Average Waiting Time: 3.00
Average Turnaround Time: 7.50
```

# Practical No. 4

## Write a Java/C++ program to simulate memory placement strategies

## 1. First Fit

## 2. Best Fit

## 3. Worst Fit

```
import java.util.Scanner;
public class MemoryAllocation {
   static void firstFit(int blockSize[], int m, int processSize[], int n) {
      int allocation[] = new int[n];
      for (int i = 0; i < n; i++) allocation[i] = -1;
      for (int i = 0; i < n; i++) {
         for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
               allocation[i] = j;
               blockSize[j] -= processSize[i];
               break;
            }
         }
      }
      System.out.println("\nFirst Fit Allocation:");
      printAllocation(processSize, allocation);
   }
   static void bestFit(int blockSize[], int m, int processSize[], int n) {
      int allocation[] = new int[n];
      for (int i = 0; i < n; i++) allocation[i] = -1;
      for (int i = 0; i < n; i++) {
         int bestIdx = -1;
         for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
               if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                  bestIdx = j;
            }
         }
         if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blockSize[bestIdx] -= processSize[i];
         }
      }
      System.out.println("\nBest Fit Allocation:");
```

```
    printAllocation(processSize, allocation);
}
static void worstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[] = new int[n];
    for (int i = 0; i < n; i++) allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        int worstIdx = -1;
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                    worstIdx = j;
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            blockSize[worstIdx] -= processSize[i];
        }
    }
    System.out.println("\nWorst Fit Allocation:");
    printAllocation(processSize, allocation);
}
static void printAllocation(int processSize[], int allocation[]) {
    System.out.println("Process No.\tProcess Size\tBlock No.");
    for (int i = 0; i < processSize.length; i++) {
        System.out.print(" " + (i + 1) + "\t\t" + processSize[i] + "\t\t");
        if (allocation[i] != -1)
            System.out.println(allocation[i] + 1);
        else
            System.out.println("Not Allocated");
    }
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter number of memory blocks: ");
    int m = sc.nextInt();
    int blockSize1[] = new int[m];
    System.out.println("Enter block sizes:");
    for (int i = 0; i < m; i++) blockSize1[i] = sc.nextInt();
    System.out.print("Enter number of processes: ");
    int n = sc.nextInt();
    int processSize[] = new int[n];
```

```
        System.out.println("Enter process sizes:");
        for (int i = 0; i < n; i++) processSize[i] = sc.nextInt();
        int blockSize2[] = blockSize1.clone();
        int blockSize3[] = blockSize1.clone();
        firstFit(blockSize1, m, processSize, n);
        bestFit(blockSize2, m, processSize, n);
        worstFit(blockSize3, m, processSize, n);
        sc.close();
    }
}
```

## Output:

```
lab@lab-VirtualBox:~/Downloads$ java MemoryAllocation
Enter number of memory blocks: 5
Enter block sizes:
100 500 200 300 600
Enter number of processes: 4
Enter process sizes:
212 417 112 426

First Fit Allocation:
Process No.     Process Size    Block No.
 1              212             2
 2              417             5
 3              112             2
 4              426             Not Allocated

Best Fit Allocation:
Process No.     Process Size    Block No.
 1              212             4
 2              417             2
 3              112             3
 4              426             5

Worst Fit Allocation:
Process No.     Process Size    Block No.
 1              212             5
 2              417             2
 3              112             5
 4              426             Not Allocated
lab@lab-VirtualBox:~/Downloads$
```

# Practical No. 5

**Write a Java Program (using OOP features) to implement paging simulation using**

**1. FIFO**

**2. Least Recently Used (LRU)**

**3. Optimal algorithm**

```
import java.util.*;
abstract class PageReplacement {
   protected int capacity;
   protected List<Integer> pages;
   protected int faults;
   public PageReplacement(int capacity) {
      this.capacity = capacity;
      this.pages = new ArrayList<>();
      this.faults = 0;
   }
   public abstract void simulate(int[] referenceString);
   public void printResult(String algoName) {
      System.out.println("\n" + algoName + " Page Replacement:");
      System.out.println("Total Page Faults: " + faults);
   }
}
// FIFO Implementation
class FIFO extends PageReplacement {
   public FIFO(int capacity) {
      super(capacity);
   }
   @Override
   public void simulate(int[] referenceString) {
      Queue<Integer> queue = new LinkedList<>();
      for (int page : referenceString) {
         if (!pages.contains(page)) {
            faults++;
            if (pages.size() < capacity) {
               pages.add(page);
               queue.add(page);
            } else {
               int removed = queue.poll();
               pages.remove((Integer) removed);
```

```
                pages.add(page);
                queue.add(page);
            }
        }
    }
    printResult("FIFO");
  }
}
// LRU Implementation
class LRU extends PageReplacement {
  public LRU(int capacity) {
    super(capacity);
  }
  @Override
  public void simulate(int[] referenceString) {
    LinkedHashMap<Integer, Integer> cache = new LinkedHashMap<>(capacity, 0.75f, true);
    for (int page : referenceString) {
      if (!cache.containsKey(page)) {
        faults++;
        if (cache.size() == capacity) {
          int lruPage = cache.entrySet().iterator().next().getKey();
          cache.remove(lruPage);
          pages.remove((Integer) lruPage);
        }
        cache.put(page, 1);
        pages.add(page);
      } else {
        cache.get(page); // just to update order
      }
    }
    printResult("LRU");
  }
}
// Optimal Implementation
class Optimal extends PageReplacement {
  public Optimal(int capacity) {
    super(capacity);
  }
  @Override
  public void simulate(int[] referenceString) {
    for (int i = 0; i < referenceString.length; i++) {
```

```
            int page = referenceString[i];
            if (!pages.contains(page)) {
                faults++;
                if (pages.size() < capacity) {
                    pages.add(page);
                } else {
                    int farthest = i + 1;
                    int pageToReplace = -1;
                    for (int p : pages) {
                        int nextUse = Integer.MAX_VALUE;
                        for (int k = i + 1; k < referenceString.length; k++) {
                            if (referenceString[k] == p) {
                                nextUse = k;
                                break;
                            }
                        }
                        if (nextUse > farthest) {
                            farthest = nextUse;
                            pageToReplace = p;
                        }
                    }
                    if (pageToReplace == -1) {
                        pageToReplace = pages.get(0);
                    }
                    pages.remove((Integer) pageToReplace);
                    pages.add(page);
                }
            }
        }
        printResult("Optimal");
    }
}
// Driver Class
public class PagingSimulation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of frames: ");
        int capacity = sc.nextInt();
        System.out.print("Enter length of reference string: ");
        int n = sc.nextInt();
        int referenceString[] = new int[n];
```

```
        System.out.println("Enter reference string:");
        for (int i = 0; i < n; i++) {
            referenceString[i] = sc.nextInt();
        }
        PageReplacement fifo = new FIFO(capacity);
        fifo.simulate(referenceString);
        PageReplacement lru = new LRU(capacity);
        lru.simulate(referenceString);
        PageReplacement optimal = new Optimal(capacity);
        optimal.simulate(referenceString);
        sc.close();
    }
}
```

## Output:

```
lab@lab-VirtualBox:~/Downloads$ javac PagingSimulation.java
lab@lab-VirtualBox:~/Downloads$ java PagingSimulation
Enter number of frames: 3
Enter length of reference string: 12
Enter reference string:
7 0 1 2 0 3 0 4 2 3 0 3

FIFO Page Replacement:
Total Page Faults: 10

LRU Page Replacement:
Total Page Faults: 9

Optimal Page Replacement:
Total Page Faults: 7
lab@lab-VirtualBox:~/Downloads$
```